Session 9: Pointers & Functions

Why Study Pointers with Functions?

In C programming, **functions and pointers work together** to:

- Improve performance
- Reduce memory usage
- Allow modification of data across functions
- Handle arrays and strings efficiently

Arrays and strings cannot be passed by value in C
They are always accessed using **pointers**

Understanding Memory Before Functions

Example:

int arr[5] = {10, 20, 30, 40, 50};

Memory layout (simplified):

Address	Value
1000	10
1004	20
1008	30
1012	40
1016	50

- arr → stores address 1000
- &arr[0] → same as arr
- arr + 1 → points to 20

Array name itself is a pointer constant

Passing Arrays to Functions (Core Concept)

What Happens When We Pass an Array?

function(arr, size);

- arr does NOT pass the whole array

- Only the **address of first element** is passed
- Function receives a pointer

Function Declaration Styles (ALL SAME)

void func(int arr[]); void

func(int arr[10]); void

func(int *arr);

✓ All three mean:
"Receive the base address of an integer array"

Why Array Size Must Be Passed?

C **does not store array size information** automatically.

So inside function:

- We only know the starting address
- We don't know where the array ends

This is impossible:

int size = sizeof(arr) / sizeof(arr[0]); // WRONG inside function

✓ Therefore, **size must be passed explicitly**

Call by Value vs Call by Reference

Call by Value (Normal Variables) void

change(int x) {   x = 100;

}

- Copy is passed
- Original value unchanged

Call by Reference (Using Pointers)

void change(int *x) {

   *x = 100;

}

- Address is passed

- Original value changes

Arrays automatically behave like call by reference

Passing Strings to Functions

Strings are **character arrays**, so rules are the same.

Example:

void display(char str[]) {

printf("%s", str);

}

Internally:

void display(char *str);

- str points to first character
- '\0' tells function where string ends

Pointer Arithmetic (Very Important)

For array: int

arr[5];

Expression Meaning arr Address of

first element arr + i Address of ith

element *(arr + i) Value of ith

element arr[i] Same as *(arr + i)

Array indexing is internally pointer arithmetic

Function Pointers (Advanced Concept)

What is a Function Pointer?

A **function pointer** stores the **address of a function**, not data.

Syntax:

return_type (*pointer_name)(parameters);

Example:

```
int add(int a, int b) {
return a + b;
}

int (*fp)(int, int) = add;
Calling: fp(2, 3);   //
returns 5
```

Used in:

- Callbacks
- Event handling
- Menu-driven programs

ACTIVITY: Finding Average of Array Elements

Problem Statement (Detailed) Write a program that:

- Accepts an array of integers
- Accepts the size of the array
- Passes both to a function
- Calculates and returns the **average**

Step-by-Step Logic

1. Receive base address of array
2. Receive size separately
3. Initialize sum = 0
4. Traverse array using pointer/index
5. Add all elements
6. Divide sum by size
7. Return average as float

Function Prototype Analysis

float findAverage(int arr[], int size); Internally

interpreted as:

float findAverage(int *arr, int size);

✓ arr → pointer to first element
✓ size → controls loop

Complete Program (Explained Line by Line)

```
#include <stdio.h>


float findAverage(int *arr, int size) {

int sum = 0;


   for(int i = 0; i < size; i++) {        sum +=

*(arr + i);  // pointer arithmetic

   }


   return (float)sum / size;

}


int main() {     int

arr[100], n;


   printf("Enter number of elements: ");

scanf("%d", &n);


   printf("Enter elements:\n");

for(int i = 0; i < n; i++) {

scanf("%d", &arr[i]);

   }
```

```
    float avg = findAverage(arr, n);
printf("Average = %.2f", avg);


    return 0;
}
```

Dry Run (Detailed)

Input: n = 4 arr =

10 20 30 40

Execution:

sum = 0 sum

= 10 sum =

30 sum = 60

sum = 100

average = 100 / 4 = 25.0

Output:

Average = 25.00

Memory Flow Diagram (Conceptual)

```
main()
 |
 |--- arr ----> [10][20][30][40]
 |--- n = 4
 |
 |--- findAverage(arr, 4)
        |
        |--- arr (pointer) → same base address
        |--- sum calculated
```

✓ No array copying
✓ Fast execution

✓ Memory efficient

Common Student Mistakes

Forgetting to pass size
Dividing int by int (no typecasting)
Assuming array is copied
Accessing out-of-bound index

Key Takeaways (Must Remember)

✓ Arrays are passed as pointers
✓ Function receives base address only
✓ Size must always be passed
✓ Pointer arithmetic = array indexing
✓ Function pointers store function addresses